# 1.2 Linux Terminal & Filesystem

### Edge AI Engineering Bootcamp | Analog Data

---

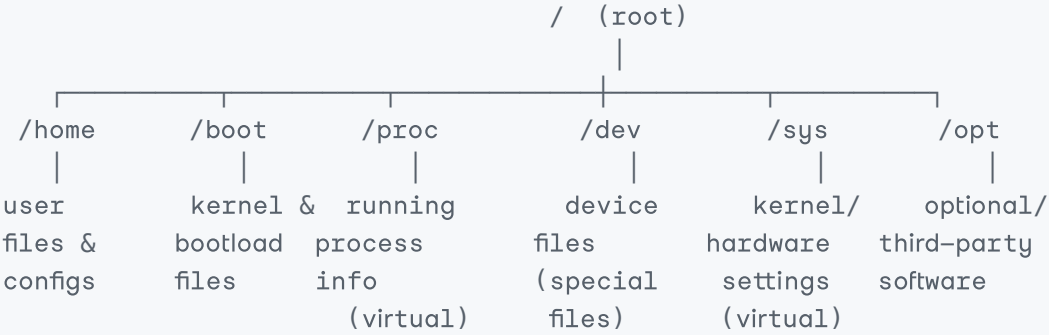
## Why This Matters

Every Raspberry Pi runs Linux under the hood. Before you can build Edge AI projects, you need to be comfortable *living* inside the Linux terminal — navigating the filesystem, finding files, controlling permissions, and managing running programs. This module builds that foundation.

---

## 1. Filesystem Hierarchy

Unlike Windows (which has `C:\`, `D:\`, etc.), Linux has **one single tree**, starting from the root `/`. Every drive, folder, and device is a branch of this same tree.




| Directory | What It Actually Contains | Real Example |
|---|---|---|
| `/home` | Personal folders for each user account. Your files, downloads, projects, `.bashrc` config, etc. live here. | `/home/pi/my_project/main.py` |
| `/boot` | Kernel image, bootloader files, and — on Raspberry Pi specifically — `config.txt` and `cmdline.txt`, which control hardware settings like camera, overclocking, and GPU memory split. | `/boot/config.txt`, `/boot/firmware/config.txt` (newer OS versions) |
| `/proc` | A **virtual** filesystem — it doesn't exist on disk at all. The kernel generates it live, in real time, to expose information about running processes and system state. | `/proc/cpuinfo` (CPU details), `/proc/meminfo` (RAM usage), `/proc/1234` (info on process ID 1234) |

| Directory | What It Actually Contains | Real Example |
|---|---|---|
| `/dev` | Represents hardware devices as files. Reading/writing to these "files" actually talks to real hardware. | `/dev/sda` (a storage disk), `/dev/ttyUSB0` (a USB serial device), `/dev/i2c-1` (I2C bus — important for sensors!) |
| `/sys` | Another **virtual** filesystem, similar to `/proc`, but focused on kernel and hardware device settings you can sometimes read *and write* to control hardware behavior. | `/sys/class/thermal/thermal_zone0/temp` (CPU temperature) |
| `/opt` | Optional, third-party, or manually-installed software that doesn't come from the standard package manager. | `/opt/vc/` (Broadcom VideoCore tools on Raspberry Pi OS) |

**Teaching point to remember:** `/proc` and `/sys` are not real files on your SD card — they're windows into the live, running kernel. If you `cat /proc/cpuinfo`, you're not reading a stored file, you're asking the kernel to generate that info *right now*.

**Try it yourself:**

```
cat /proc/cpuinfo          # see your Pi's CPU details
cat /proc/meminfo          # see RAM usage details
ls /dev/                   # see all recognized devices
cat /boot/config.txt       # view your Pi's hardware config
```

---

## 2. Essential Commands

### `find` — search for files by name, type, size, date, etc.

```
find /home/pi -name "*.py"              # find all Python files under /home/pi
find / -type d -name "models"           # find directories named "models"
find . -size +100M                      # find files bigger than 100MB in current
folder
find . -mtime -1                        # find files modified in the last 1 day
```

### `grep -r` — search *inside* file contents, recursively

```
grep -r "import cv2" .                  # find every file using OpenCV, in current
folder & subfolders
grep -rn "TODO" .                       # same, but show line numbers (-n)
grep -ril "error" /var/log/             # search case-insensitively (-i), list
filenames only (-l)
```

### `awk` — process text column by column

`awk` is built for working with structured text (like columns in a log file or CSV).

```
awk '{print $1}' file.txt                # print the first column of every line
df -h | awk '{print $1, $5}'               # print disk name and usage % from df
output
awk -F',' '{print $2}' data.csv            # use comma as separator, print 2nd
column
```

### `sed` — find and replace text in a stream

```
sed 's/old/new/' file.txt                  # replace first "old" with "new" on each
line
sed 's/old/new/g' file.txt                 # replace ALL occurrences (g = global)
on each line
sed -i 's/DEBUG=False/DEBUG=True/' app.py  # edit the file directly (-i = in-place)
```

### `tail -f` — watch a file live as it grows (essential for logs)

```
tail -f /var/log/syslog                    # watch system log update in real time
tail -n 50 app.log                         # show just the last 50 lines
tail -f training.log | grep "epoch"         # watch only lines containing "epoch"
```

### `tee` — write output to a file AND show it on screen at the same time

```
ls -la | tee output.txt                      # see the output AND save it to
output.txt
some_command | tee -a log.txt                 # append (-a) instead of overwrite
sudo command | tee /etc/some_config.conf     # useful with sudo when redirecting
normally fails
```

**Combining commands (this is where Linux gets powerful):**

```
find . -name "*.log" | xargs grep -l "ERROR"     # find all .log files that contain
"ERROR"
ps aux | grep python | awk '{print $2}'           # get just the PIDs of running
python processes
```

---

## 3. Permissions

Linux is a **multi-user system** — every file has an owner, a group, and a set of permission rules controlling who can read, write, or execute it.

```
-rwxr-xr--  1  pi  users  1024  Jul 1 18:00  script.sh
|└─┘└─┘└─┘
|  |   |   |
|  |   |   └── permissions for OTHERS  (r-- = read only)
|  |   └────── permissions for GROUP   (r-x = read + execute)
```

```
|  └──────────── permissions for OWNER   (rwx = read + write + execute)
└──────────────── file type  (- = regular file,  d = directory)

  owner  ──────────────────────────────────────────────────► pi
  group  ──────────────────────────────────────────────────► users
  links  ──────────────────────────────────────────────────► 1
```

**Permission values (used with `chmod`):**

| Permission | Symbol | Number |
|---|---|---|
| Read | `r` | 4 |
| Write | `w` | 2 |
| Execute | `x` | 1 |
| None | `-` | 0 |

Add the numbers together for each group (owner / group / others):

```
chmod 755 script.sh     # owner: rwx (7) | group: r-x (5) | others: r-x (5)
chmod 644 config.txt     # owner: rw- (6) | group: r-- (4) | others: r-- (4)
chmod +x script.sh        # just add execute permission for everyone
chmod u+w file.txt         # add write permission for the owner (u) only
```

**`chown` — change who owns a file:**

```
sudo chown pi:pi file.txt          # set owner=pi, group=pi
sudo chown -R pi:pi /home/pi/project/    # apply recursively (-R) to a whole
folder
```

**`sudo` — temporarily act as the superuser (root)**

Most system-level changes (installing software, editing system configs, restarting services) require root privileges.

```
sudo apt update              # run a command with root privileges
sudo -i                      # open a full root shell (use carefully!)
sudo -u pi some_command       # run a command AS a specific user (pi)
```

**`/etc/sudoers` — controls who is allowed to use `sudo`, and what they can run**

Never edit this file directly with a normal text editor — always use:

```
sudo visudo
```

`visudo` checks your syntax before saving, preventing you from accidentally locking yourself out of `sudo` entirely with a typo.

**Example line inside `/etc/sudoers`:**

```
pi ALL=(ALL) NOPASSWD: ALL
```

This means the user `pi` can run any command as any user, without being asked for a password. Useful for automation, but a security risk if misused.

---

## 4. Process Management

Every running program on Linux is a **process**, identified by a unique **PID** (Process ID).

### `htop` — interactive, real-time process viewer

```
htop
```

Shows live CPU/RAM usage per core, a scrollable list of all processes, and lets you sort, search, and kill processes directly from the interface (press `F9` to kill, `q` to quit).

*(If not installed: `sudo apt install htop`)*

### `kill` — stop a process by its PID

```
kill 1234                # politely ask process 1234 to stop (SIGTERM)
kill -9 1234              # force-kill immediately, no cleanup (SIGKILL) — last
resort
kill -l                   # list all available signals
```

**Finding the PID first:**

```
ps aux | grep python        # list processes, filter for "python"
pgrep -f "train_model.py"    # get PID directly by matching command text
```

### `nice` / `renice` — control how much CPU priority a process gets

Linux uses a "niceness" value from **-20 (highest priority)** to **19 (lowest priority)**. A process with a *higher* nice value is more "nice" to other processes — it steps back and lets them run first.

```
nice -n 10 python train_model.py       # start a process with LOWER priority
(nice=10)
nice -n -5 python realtime_inference.py  # start with HIGHER priority (needs sudo for
negative values)
renice -n 15 -p 1234                    # change priority of an ALREADY RUNNING
process (PID 1234)
```

**Real example for Edge AI work:** If you're running a heavy model training job in the background but still need the Pi responsive for SSH and monitoring, start the training with `nice -n 10` so it doesn't starve the rest of the system.

---

## 5. `systemctl` — Managing System Services

Linux uses **systemd** to manage background services (things that start automatically and run continuously, like SSH, networking, or your own custom Edge AI scripts).

```
┌──────────────────────────────────────────────────────┐
│                      systemd                         │
│                                                      │
│   ssh.service           → running   (enabled at boot)   │
│   nginx.service         → stopped   (disabled)          │
│   my_ai_app.service     → running   (enabled at boot)   │
└──────────────────────────────────────────────────────┘
```

| Command | What It Does |
|---|---|
| `sudo systemctl start <service>` | Start a service right now |
| `sudo systemctl stop <service>` | Stop a running service |
| `sudo systemctl restart <service>` | Stop then start again (common after config changes) |
| `sudo systemctl enable <service>` | Make the service auto-start on every boot |
| `sudo systemctl disable <service>` | Stop it from auto-starting on boot |
| `systemctl status <service>` | Check if it's running, and see recent log lines |
| `journalctl -u <service>` | View the full log history for a specific service |
| `journalctl -u <service> -f` | Follow the log live, like `tail -f` |

**Practical examples:**

```
sudo systemctl status ssh              # check if SSH is running
sudo systemctl enable ssh               # make sure SSH starts automatically on
every boot
sudo systemctl restart networking        # restart networking after a config change
journalctl -u my_ai_app -f               # watch your custom service's logs live
```

**Why this matters for Edge AI projects:** Once you build a Python script that runs your camera + AI pipeline, you'll want it to **start automatically every time the Pi boots**, and to **restart itself if it crashes**. This is done by creating your own `systemd` service file (we'll cover writing one in a later module) and using these exact commands to control it.

---

## Quick Recap

1. Linux has **one filesystem tree** starting at `/` — know what `/home`, `/boot`, `/proc`, `/dev`, `/sys`, and `/opt` are actually for.
2. `find`, `grep -r`, `awk`, `sed`, `tail -f`, and `tee` are your core toolkit for searching, filtering, and monitoring text and files.

3. Permissions ( `chmod` , `chown` ) and `sudo` / `/etc/sudoers` control who can do what — always edit sudoers with `visudo` .
4. `htop` , `kill` , `nice` , and `renice` let you monitor and control running processes and their CPU priority.
5. `systemctl` (with `journalctl` ) manages background services — critical for making your Edge AI programs run automatically and reliably.

---

## Self-Check Questions

1. Why is `/proc/cpuinfo` not a "real" file stored on your SD card?
2. What does `chmod 755` actually set, broken down by owner/group/others?
3. What's the difference between `kill 1234` and `kill -9 1234` ?
4. If you wanted your custom Edge AI script to start automatically every time the Pi reboots, which two `systemctl` commands would matter most?
5. Why should you always use `visudo` instead of opening `/etc/sudoers` with a regular text editor?
6. Write a single command that finds all `.log` files in the current folder and shows only the lines containing the word "ERROR".